

Projet :

Développement de logiciel (AKSUS_LOG) en version application de traitement et de gestion de données (archives et données courantes) d'entreprises, d'administration, de cabinets de notaires, avocats, instances judiciaires et juridiques, d'administration publique, etc.

Déployable automatiquement avec plusieurs utilisateurs en administration (5 à 77 utilisateurs en réseau) contre licence renouvelable avec des options gratuite et payante chaque 3ans.

Logiciel en version application Web et mobile produit de la société **Afrik Data and Technology sarl** du Bénin.

Stack proposée

- Backend : Node.js + Express (ou NestJS) avec TypeScript
- Frontend : React (Next.js) PWA
- Base de données : PostgreSQL 15
- Cache : Redis
- Stockage objet : MinIO (on-premise compatible S3)
- File d'attente : BullMQ (basé sur Redis)
- Signature électronique : Intégration PKI avec openssl / API Tiers de confiance
- Paiement mobile : CinetPay, Fedapay, MTN Mobile Money (via API agrégateur)
- Conteneurisation : Docker, docker-compose

1. Configuration de l'environnement (docker-compose.yml)

```
``yaml
```

```
Version : '3.8'
```

Services :

Postgres :

Image : postgres:15

Container_name : archibenin-db

Environment :

POSTGRES_USER : archibenin

POSTGRES_PASSWORD : changeme

POSTGRES_DB : archibenin

Volumes :

- Pgdata:/var/lib/postgresql/data

Ports :

- « 5432 :5432 »

Redis :

Image : redis :7-alpine

Container_name : archibenin-redis

Ports :

- « 6379 :6379 »

Minio :

Image : quay.io/minio/minio

Container_name : archibenin-storage

Environment :

MINIO_ROOT_USER : minioadmin

MINIO_ROOT_PASSWORD : minioadmin

Volumes :

- Minio_data:/data

Command : server /data --console-address « :9001 »

Ports :

- « 9000 :9000 »
- « 9001 :9001 »

Api :

Build : ./backend

Container_name : archibenin-api

Env_file :

- ./backend/.env

Volumes :

- ./backend:/app

- /app/node_modules

Ports :

- « 3000 :3000 »

Depends_on :

- postgres

- redis

- minio

Worker :

Build : ./backend

Container_name : archibenin-worker

Command : npm run worker

Env_file :

- ./backend/.env

Depends_on :

- postgres

- redis

- minio

Frontend :

Build : ./frontend

Container_name : archibenin-web

Env_file :

- ./frontend/.env

Ports :

- « 8080 :3000 »

Depends_on :

- Api

Volumes :

Pgdata :

Minio_data :

...

2. Structure du projet (monorepo)

...

Archibenin-app/

```
└─ backend/
|   └─ src/
|       └─ modules/
|           └─ auth/          # Authentification, JWT, OTP
|           └─ users/        # Gestion utilisateurs
|           └─ coffre/       # Coffre-fort numérique
|           └─ box/          # Box stockage physique
|           └─ numerisation/  # Numérisation certifiée
|           └─ externalisation/ # Sauvegardes externalisées
|           └─ marchespublics/ # Suivi marchés publics
|           └─ paiement/     # Paiement mobile
|           └─ blockchain/    # Traçabilité blockchain
|           └─ legal/         # Référentiel juridique
|           └─ common/        # Middlewares, guards, helpers
|           └─ config/        # Configuration centralisée
|           └─ main.ts
|           └─ prisma/        # ORM (Prisma)
|           └─ schema.prisma
|           └─ .env.example
|           └─ Dockerfile
|           └─ package.json
|           └─ frontend/
|           └─ src/
```

```

| | └─ components/
| | └─ pages/
| | └─ services/
| | └─ i18n/          # Français (support multilingue)
| └─ public/
| └─ .env.example
| └─ Dockerfile
└─ package.json
└─ docker-compose.yml
└─ README.md
...

```

3. Extraits de code clés

3.1 Schéma Prisma (backend/prisma/schema.prisma)

3.2

```
``prisma
```

```
Generator client {
```

```
  Provider = « prisma-client-js »
```

```
}
```

```
Datasource db {
```

```
  Provider = « postgresql »
```

```
  url      = env(« DATABASE_URL »)
```

```
}
```

```
Model User {
```

```
  Id      String  @id @default(uuid())
```

```
  Email   String  @unique
```

```
  Phone   String ?
```

```
  Password String
```

```

Role    Role    @default(PME)

Company  Company ? @relation(fields : [companyId], references : [id])

companyId String ?

createdAt DateTime @default(now())
}

```

```

Model Document {

    Id        String    @id @default(uuid())

    Title     String

    Type      DocumentType

    hashSha256 String    // empreinte pour intégrité

    archivedAt DateTime  @default(now())

    metadata  Json ?

    ownerId   String

    owner     User      @relation(fields : [ownerId], references : [id])

    boxId     String ?

    box       Box ?     @relation(fields : [boxId], references : [id])

    accessLogs AccessLog[]

    signatures Signature[]
}

```

```

Model Box {

    Id        String    @id @default(uuid())

    Barcode   String    @unique

    Location  String

    Status    BoxStatus @default(ACTIVE)

    Documents Document[]

    destructionDate DateTime ?
}

```

```

Model AccessLog {

```

```

    Id      String @id @default(uuid())
    documentId String
    document Document @relation(fields : [documentId], references : [id])
    userId   String
    accessedAt DateTime @default(now())
    action   ActionType
    ip       String
}

```

```

Model Signature {
    Id      String @id @default(uuid())
    documentId String
    document Document @relation(fields : [documentId], references : [id])
    signedBy String
    signedAt DateTime @default(now())
    certificate String // certificat qualifié
    signature String // signature électronique
}

```

```

Enum Role {
    ADMIN
    ARCHIVISTE
    PME
    AUDITEUR
}

```

```

Enum DocumentType {
    CONTRAT
    FACTURE
    MARCHE_PUBLIC
    PIECE_COMPTABLE
}

```

```
AUTRE  
}
```

```
Enum BoxStatus {  
  ACTIVE  
  DESTROYED  
  TRANSFERRED  
}
```

```
Enum ActionType {  
  VIEW  
  DOWNLOAD  
  DELETE  
  MODIFY  
}  
...
```

3.3 Module Coffre-fort – Dépôt avec horodatage et signature

3.4

```
``typescript
```

```
// backend/src/modules/coffre/coffre.service.ts
```

```
Import { createHash } from 'crypto' ;
```

```
Import { S3Client, PutObjectCommand } from '@aws-sdk/client-s3' ;
```

```
Import { PrismaService } from '../prisma.service' ;
```

```
Import { v4 as uuid } from 'uuid' ;
```

```
Const s3 = new S3Client({  
  Endpoint : process.env.MINIO_ENDPOINT,  
  Region : 'us-east-1',  
  Credentials : {  
    accessKeyId : process.env.MINIO_ACCESS_KEY,
```

```
secretAccessKey : process.env.MINIO_SECRET_KEY,  
},  
forcePathStyle : true,  
});
```

```
Export class CoffreService {
```

```
  Constructor(private prisma : PrismaService) {}
```

```
  Async deposerDocument(file : Express.Multer.File, userId : string, metadata : any) {
```

```
    // 1. Calcul de l'empreinte SHA-256
```

```
    Const hash = createHash('sha256').update(file.buffer).digest('hex');
```

```
    // 2. Stockage dans MinIO (bucket 'coffre')
```

```
    Const key = `documents/${uuid()}-${file.originalname}`;
```

```
    Await s3.send(new PutObjectCommand({
```

```
      Bucket : 'coffre',
```

```
      Key : key,
```

```
      Body : file.buffer,
```

```
      ContentType : file.mimetype,
```

```
      Metadata : { sha256 : hash },
```

```
    }));
```

```
    // 3. Horodatage qualifié (appel à un service externe ici simplifié)
```

```
    Const horodatage = new Date().toISOString(); // en vrai, token RFC 3161
```

```
    // 4. Enregistrement en base
```

```
    Const doc = await this.prisma.document.create({
```

```
      Data : {
```

```
        Title : file.originalname,
```

```
        Type : metadata.type || 'AUTRE',
```

```
        hashSha256 : hash,
```

```

        ownerId : userId,

        metadata : {
            ...metadata,
            minioKey : key,
            horodatage,
        },
    },
});

// 5. Journal d'audit
Await this.prisma.accessLog.create({
    Data : {
        documentId : doc.id,
        userId,
        action : 'VIEW', // première consultation par le déposant
        ip : '0.0.0.0',
    },
});

Return doc ;
}
}
...

```

3.3 Intégration paiement mobile (ex : CinetPay)

```

```typescript
// backend/src/modules/paiement/paiement.service.ts

Import axios from 'axios' ;

Export class PaiementService {

```

```
Async initierPaiement(transactionId : string, montant : number, telephone : string, operateur : 'mtn' | 'moov') {
```

```
 Const payload = {
 Apikey : process.env.CINETPAY_API_KEY,
 Site_id : process.env.CINETPAY_SITE_ID,
 Transaction_id : transactionId,
 Amount : montant,
 Currency : 'XOF',
 Channels : 'MOBILE_MONEY',
 Customer_phone : telephone,
 Customer_name : 'Acheteur DAO',
 Description : 'Achat dossier appel d\'offres',
 Notify_url : `${process.env.BASE_URL}/api/paiement/callback`,
 Return_url : `${process.env.FRONTEND_URL}/paiement/succes`,
 };
```

```
 Const response = await axios.post(
 'https://api-checkout.cinetpay.com/v1/payment',
 Payload,
 { headers : { 'Content-Type' : 'application/json' } }
);
 Return response.data ;
}
```

```
Async verifierStatut(transactionId : string) {
 Const response = await axios.post('https://api-checkout.cinetpay.com/v1/payment/check', {
 Apikey : process.env.CINETPAY_API_KEY,
 Site_id : process.env.CINETPAY_SITE_ID,
 Transaction_id : transactionId,
 });
 Return response.data ;
}
```

```
}
}
...
```

### 3.5 Suivi des marchés publics – Agrégateur AO

#### 3.6

```
```typescript
```

```
// backend/src/modules/marchespublics/ao.service.ts
```

```
Import * as cheerio from 'cheerio' ;
```

```
Import axios from 'axios' ;
```

```
Export class AOService {
```

```
  Async recupererAppelsDOffres() : Promise<any[]> {
```

```
    // En production, utiliser un flux RSS officiel ou API
```

```
    Const url = 'https://www.marches-publics.bj/avis' ; // fictif
```

```
    Const { data } = await axios.get(url) ;
```

```
    Const $ = cheerio.load(data) ;
```

```
    Const avis : any[] = [] ;
```

```
    $(''.avis-item').each((i, el) => {
```

```
      Avis.push({
```

```
        Titre : $(el).find(''.titre').text(),
```

```
        dateLimite : $(el).find(''.date').text(),
```

```
        reference : $(el).attr('data-ref'),
```

```
        lien : $(el).find('a').attr('href'),
```

```
        autoriteContractante : $(el).find(''.autorite').text(),
```

```
      }) ;
```

```
    }) ;
```

```
    Return avis ;
```

```
  }
```

```
}
```

```
...
```

3.5 Authentification OTP (via SMS – intégration opérateur local)

```
```typescript
```

```
// backend/src/modules/auth/auth.controller.ts
```

```
Import { Controller, Post, Body } from '@nestjs/common' ;
```

```
Import { AuthService } from './auth.service' ;
```

```
@Controller('auth')
```

```
Export class AuthController {
```

```
 Constructor(private authService : AuthService) {}
```

```
 @Post('send-otp')
```

```
 Async sendOTP(@Body('phone') phone : string) {
```

```
 Await this.authService.generateAndSendOTP(phone) ;
```

```
 Return { message : 'OTP envoyé' } ;
```

```
 }
```

```
 @Post('login')
```

```
 Async login(@Body() body : { phone : string ; otp : string }) {
```

```
 Const user = await this.authService.validateOTP(body.phone, body.otp) ;
```

```
 Const token = this.authService.generateJWT(user) ;
```

```
 Return { access_token : token } ;
```

```
 }
```

```
}
```

```
```
```

3.7 Middleware de traçabilité blockchain simplifiée

3.8

```
```typescript
```

```
// backend/src/common/blockchain-logger.ts
```

```

Import { createHash } from 'crypto' ;

// Simule un ancrage blockchain (en vrai, utiliser Hyperledger ou Ethereum privé)
Export class BlockchainLogger {
 Private chain : string[] = [] ;

 Log(event : any) {
 Const block = {
 Timestamp : new Date().toISOString(),
 Data : event,
 previousHash : this.chain.length ? this.chain[this.chain.length - 1] : '0',
 } ;
 Const hash = createHash('sha256').update(JSON.stringify(block)).digest('hex') ;
 This.chain.push(hash) ;
 // Stocker dans un fichier/DB pour audit
 Return hash ;
 }
}
...

```

#### 4. Variables d'environnement (backend/.env.example)

```

``ini
Base de données
DATABASE_URL=postgresql://archibenin :changeme@postgres :5432/archibenin

MinIO
MINIO_ENDPOINT=http://minio :9000
MINIO_ACCESS_KEY=minioadmin
MINIO_SECRET_KEY=minioadmin

```

# JWT

JWT\_SECRET=une\_cle\_secrete\_longue

# SMS OTP (exemple via l'API d'un opérateur béninois)

SMS\_API\_KEY=xxx

SMS\_SENDER=ARCHIBENIN

# Paiement mobile

CINETPAY\_API\_KEY=xxx

CINETPAY\_SITE\_ID=123456

FEDAPAY\_API\_KEY=xxx

# Signature électronique (Tiers de confiance)

SIGNATURE\_API\_URL=https://pki-cert.bj/api/sign

SIGNATURE\_API\_KEY=xxx

# Frontend URL

FRONTEND\_URL=http://localhost :8080

BASE\_URL=http://localhost :3000

# Notification violation de données (RGPD/APDP)

[DPO\\_EMAIL=dpo@archibenin.bj](mailto:dpo@archibenin.bj)

...

## 5. Lancement du développement

```
```bash
```

Cloner le projet

Git clone ... && cd archibenin-app

Copier les fichiers .env

Cp backend/.env.example backend/.env

Cp frontend/.env.example frontend/.env

Lancer les conteneurs

Docker-compose up -d

Initialiser la base de données (migrations Prisma)

Docker-compose exec api npx prisma migrate dev --name init

Accéder :

Frontend : <http://localhost:8080>

MinIO Console : <http://localhost:9001>

API : <http://localhost:3000>

...